

NATIONAL ECONOMICS UNIVERSITY
NEU – COLLEGE OF TECHNOLOGY



TRAVELING SALESMAN PROBLEM

GROUP 4.4: NGUYỄN THỊ NGỌC KHÁNH
CHU KHÁNH LY
ĐẶNG NGUYỄN THẢO MY

Class: DS67B

Supervising lecture: Assoc. Prof., Dr. Hà Minh Hoàng

Hanoi, May 2026

Declaration

We hereby declare that all the contents contained in this project entitled “TRAVELLING SALES-MAN PROBLEM” are the result of our own independent work and research effort. All analyses, discussions, data collection, and conclusions in this report were carried out and completed by the members of our group, without any references from other authors.

We firmly affirm that this project has not been submitted for evaluation in any other program or for any academic purpose or qualification at the National Economics University.

The work was conducted independently at the National Economics University, with all collaborations clearly stated in the text and Acknowledgements.

Group 4.4

May 2026

Acknowledgements

We would like to express our sincere gratitude to all individuals and institutions whose support contributed to the successful completion of this project.

First, we are deeply thankful to our supervisor, Assoc. Prof. Dr. Hà Minh Hoàng, for their expert guidance, constructive feedback, and continuous encouragement throughout the development of this study. Their insights were fundamental in shaping our research direction and ensuring the quality of our work. We would also like to acknowledge the help of members from FPC for teaching us the fundamental knowledge. Also, National Economics University, particularly the Data Science in Finance and E-commerce program for providing essential resources, academic support, and a positive learning environment. Their dedication played an important role in facilitating our progress.

Our appreciation also extends to our classmates and peers, whose valuable discussions, collaboration, and shared knowledge greatly assisted us in overcoming challenges and refining our methodology. Lastly, we would like to express our heartfelt gratitude to our families for their unwavering support, patience, and encouragement. Their belief in our abilities was a constant source of motivation during the entire project.

Abstract

The Traveling Salesman Problem (TSP) is a canonical NP-hard combinatorial optimization problem with substantial practical importance in logistics and supply-chain management. Because exact solvers are infeasible at industrial scale, the dominant approach in practice is to combine a constructive heuristic, which builds an initial feasible tour from scratch, with a local-search procedure, which iteratively refines it. This report implements and empirically evaluates four such pipelines — Farthest Insertion + Swap (the assigned main method), Farthest Insertion + 2-opt, Nearest Neighbor + Swap, and Nearest Neighbor + 2-opt — on a benchmark suite of eight TSPLIB instances covering four edge-weight conventions (EUC_2D, CEIL_2D, GEO, EXPLICIT) and instance sizes from 14 to 7,397 cities. Each algorithm is implemented with neighbour-pruned First-Improvement acceptance and double-bridge restart perturbation; convex-hull multi-start is used to reduce dependence on the choice of initial city; and every produced tour is verified valid by a dedicated validator.

The results show that Farthest Insertion + 2-opt is the strongest pipeline on six of the eight instances, while the assigned main pipeline (FI + Swap) is competitive on small instances but uniformly second-best on the medium range. Two findings are worth highlighting: first, the choice of local-search move (Swap vs. 2-opt) generally has a larger impact on the final tour length than the choice of constructive method, with the difference growing as the initial tour gets worse; and second, on one instance (d2103.tsp) the ranking inverts completely, with NN-based pipelines outperforming FI-based ones by more than fourteen percentage points — a structural anomaly that highlights the absence of a universally superior heuristic and motivates the diverse benchmark suite adopted here.

Contents

List of Figures	vi
List of Tables	vii
1 Introduction	1
1.1 Background and Motivation	1
1.2 Approach	1
1.3 Objectives and Scope	2
2 Literature Review	4
2.1 Constructive Heuristics	4
2.1.1 Main Method – Farthest Insertion	4
2.1.2 Reference Baseline – Nearest Neighbor	4
2.2 Local Search Methods	4
2.2.1 Main Method – Swap	5
2.2.2 Reference Upper-Bound – 2-opt	5
3 Dataset & Methodology	6
3.1 Benchmark Suite	6
3.1.1 Edge-Weight Types Used	6
3.2 Distance Matrix Construction	7
3.2.1 TSPLIB Parser	7
3.2.2 Multi-Type Distance Construction	8
3.2.3 Helper Functions	10
3.3 Main Methods	11
3.3.1 Farthest Insertion with Convex-Hull Multi-Start	11
3.3.2 Swap Local Search with Double-Bridge Restarts	12
3.4 Reference Methods	13
3.4.1 Nearest Neighbor with Multi-Start	13
3.4.2 2-opt Local Search with Double-Bridge Restarts	14
3.5 Validation Procedure	16

3.6	Experimental Protocol	17
4	Experimental Results and Analysis	18
4.1	Experimental Setup	18
4.1.1	Hardware and Software Environment	18
4.1.2	Datasets	18
4.1.3	Algorithm Parameters	18
4.1.4	Seed Protocol	19
4.1.5	Correctness Validation	19
4.2	Constructive Stage Comparison: Farthest Insertion vs. Nearest Neighbor	19
4.2.1	Solution Quality	19
4.2.2	Runtime	20
4.3	Local Search Effectiveness: Improvement Analysis	20
4.3.1	Swap on Farthest Insertion	21
4.3.2	2-opt on Nearest Neighbor	21
4.3.3	Comparative Observation	21
4.4	Full Benchmark of 4 Algorithm Pairs	22
4.4.1	Solution Quality	22
4.4.2	Runtime	22
4.4.3	Time and Quality Trade-off Analysis	23
4.5	Cross-Dataset Analysis	23
4.5.1	Scaling with Instance Size	23
4.5.2	Effect of Edge-Weight Type	23
4.5.3	Trade-off Between Time and Quality	23
4.6	Personal Perspectives	23
4.7	Summary of Experimental Findings	24
5	Conclusion	25
5.1	Summary of Findings	25
5.2	Lessons Learned	25
5.2.1	On Algorithmic Complexity	25
5.2.2	On Heuristic Design	26
5.3	Practical Applications in Logistics and Supply Chain	26
5.4	Limitations	26
5.5	Concluding Remarks	27
	Bibliography	28

List of Figures

3.1	Code snippet: <code>read_tsp()</code> – TSPLIB file parser.	8
3.2	Code snippet: <code>read_tsp()</code> – return dictionary.	8
3.3	Code snippet: <code>build_dist_matrix()</code> – explicit case.	9
3.4	Code snippet: <code>build_dist_matrix()</code> – coordinate-based cases.	10
3.5	Code snippet: helper functions <code>total_distance</code> , <code>is_valid_tour</code> , <code>calc_gap</code>	10
3.6	Code snippet: <code>convex_hull_starts()</code>	11
3.7	Code snippet: <code>greedy_farthest_insertion()</code> – setup and start selection.	12
3.8	Code snippet: <code>greedy_farthest_insertion()</code> – main insertion loop.	12
3.9	Code snippet: <code>double_bridge()</code> perturbation.	13
3.10	Code snippet: <code>nearest_neighbor()</code> – setup.	13
3.11	Code snippet: <code>nearest_neighbor()</code> – main loop.	14
3.12	Code snippet: <code>two_opt_local_search()</code> – setup.	15
3.13	Code snippet: <code>two_opt_local_search()</code> – inner loop.	16
3.14	Code snippet: <code>is_valid_tour()</code> validator.	16
3.15	Code snippet: <code>get_seeds(n)</code> – adaptive seed selection.	17

List of Tables

3.1	Benchmark suite used in this study.	6
3.2	Edge-weight conventions in the benchmark suite.	7
4.1	FI vs. NN: Gap and Runtime Across 8 Datasets.	19
4.2	Local Search Improvement. Δ (pp) = improvement in percentage points.	20
4.3	Gap (%) for All 4 Pairs Across 8 Datasets. Bold = best result per row.	22
4.4	Average Runtime per Seed (ms) for All 4 Pairs.	22

CHAPTER 1 Introduction

1.1. Background and Motivation

The Travelling Salesman Problem (TSP) is one of the most extensively studied combinatorial optimization problems in computer science and operations research. In its simplest formulation, TSP asks for a Hamiltonian cycle of minimum total weight in a weighted graph — that is, a route that visits every city exactly once and returns to the starting point with the smallest possible total distance. Despite the deceptive simplicity of its statement, TSP has been shown to be NP-hard, meaning no polynomial-time algorithm is known to solve it optimally unless $P = NP$. Consequently, for instances of practical size, ranging from thousands to millions of nodes, finding the exact optimum is computationally infeasible, and one must rely on approximation algorithms and heuristic methods.

The significance of TSP, however, extends far beyond its theoretical interest. The problem arises directly in a wide range of industrial applications: last-mile delivery route optimization at logistics giants, drill-path scheduling for printed circuit board (PCB) manufacturing, gene-sequencing order in bioinformatics, and crew or vehicle scheduling for public transportation. In modern supply chain management, where transportation costs constitute a substantial portion of total operational expenditure, even a small percentage improvement in TSP solution quality can translate into multi-million-dollar annual savings. For this reason, TSP is widely regarded as one of the most practically valuable problems in the field of Logistics and Supply Chain Management.

1.2. Approach

To tackle TSP at scale, the research community has developed three broad classes of methods:

- (i) **Exact algorithms** such as Branch-and-Bound and Branch-and-Cut, which are tractable only for instances with no more than a few hundred nodes.
- (ii) **Constructive heuristics**, which build a tour greedily from scratch, such as Nearest Neighbor, Nearest Insertion, Farthest Insertion, and Cheapest Insertion.
- (iii) **Local search algorithms**, which iteratively improve an existing tour through local mod-

ifications; examples include 2-opt, 3-opt, and simpler Swap.

A widely adopted and effective strategy in practice is to combine a constructive heuristic to produce an initial feasible tour with a local search procedure to refine it. This is precisely the approach adopted in the present project.

The main method studied in this report is the pipeline from Farthest Insertion (FI) to Swap Local Search. Farthest Insertion was chosen for the constructive phase because it prioritizes building the outer convex structure of the tour early in the process. This strategy tends to produce higher-quality initial tours compared to simpler greedy methods, especially on large-scale instances with spatially dispersed cities. As required by the assignment, Swap Local Search was implemented as the improvement heuristic to refine the solution obtained from Farthest Insertion.

To place the main result in context, two reference algorithms are additionally implemented and reported alongside the main pipeline:

- **Nearest Neighbor (NN):** a weaker constructive heuristic, included as the baseline against which FI is measured.
- **2-opt:** a stronger local-search move, included as an upper-reference that indicates how much further improvement a more sophisticated move could extract from the same initial tour.

The reference algorithms are not the focus of the report; they exist solely to give meaning to the numbers produced by the main FI+Swap pipeline.

1.3. Objectives and Scope

This project is evaluated on a diverse benchmark suite consisting of eight TSPLIB instances, ranging in size from 14 to 7,397 cities and covering multiple edge-weight types (GEO, EXPLICIT, EUC_2D, and CEIL_2D). While experiments are conducted across all instances to ensure comprehensive analysis, particular emphasis is placed on the largest and most challenging instance: `pla7397.tsp` (7,397 cities). This instance, derived from a real-world PCB drilling task, is large enough to reveal the practical limitations of purely greedy approaches and to make the computational cost of local search nontrivial.

The primary objective is to implement, analyze, and validate the required Farthest Insertion + Swap Local Search pipeline. The secondary objective is to compare this main pipeline with a simpler constructive heuristic (Nearest Neighbor) and a stronger local search method (2-opt). Specifically, the project fulfills the following tasks:

- Analyze the datasets and efficiently construct distance matrices for all instances.

- Implement the main pipeline: Farthest Insertion followed by Swap Local Search (First-Improvement).
- Implement reference algorithms (Nearest Neighbor and 2-opt) for comparison.
- Develop a validator to ensure all tours are correct and feasible.
- Report tour lengths, optimality gaps, and runtimes while analyzing the trade-off between solution quality and computational time.

CHAPTER 2 Literature Review

2.1. Constructive Heuristics

Constructive heuristics build a feasible tour from scratch through a sequence of greedy decisions. Two members of this family are used in this project: Farthest Insertion as the main method and Nearest Neighbor as a baseline reference.

2.1.1. Main Method – Farthest Insertion

The Insertion subfamily of constructive heuristics maintains a partial tour and grows it one city at a time. Among them, Farthest Insertion (FI) is adopted as the main constructive method in this project. At each step, FI selects the unvisited city whose minimum distance to the current tour is largest and then inserts it at the position that minimises the resulting increase in tour length. The effect is to fix the convex outline of the tour early on so that peripheral cities are not left to be inserted at high cost in the final iterations. This property makes FI a natural choice for geographically distributed instances.

2.1.2. Reference Baseline – Nearest Neighbor

The Nearest Neighbor (NN) algorithm is the simplest constructive heuristic for TSP: starting at an arbitrary city, the salesman repeatedly visits the closest unvisited city until all cities have been included and finally returns to the origin. NN runs in $O(n^2)$ time and is intuitive, but it has no constant-factor approximation guarantee. NN tours can be made arbitrarily long relative to insertion-based heuristics on adversarial inputs. In this report, NN is implemented for the sole purpose of providing a baseline: by reporting NN's tour length alongside FI's, the report makes explicit how much of the final quality is attributable to the more careful constructive strategy.

2.2. Local Search Methods

Local-search algorithms iteratively improve an existing tour by applying small modifications, called moves, that decrease the total length, terminating at a local optimum in which no allowed move yields any improvement. Two moves are studied in this project: Swap as the main method and 2-opt as a strong reference.

2.2.1. Main Method – Swap

The Swap (or 2-exchange) move exchanges the positions of two cities in the tour, modifying four edges symmetrically. Swap is conceptually simple to implement, but it is strictly weaker than the move described next: in particular, it cannot, in general, untangle a pair of crossing edges, which are the most common visible signature of a sub-optimal Euclidean tour. Swap is adopted as the main local-search method in accordance with the assignment specification, and because it cleanly isolates the essential mechanics of local search — neighborhood, delta evaluation, first-improvement acceptance, and convergence to a local optimum — without the implementation overhead of stronger moves.

2.2.2. Reference Upper-Bound – 2-opt

The 2-opt removes two non-adjacent edges from the tour and reconnects the resulting path segments in the unique alternative way, effectively reversing the segment of the tour that lies between the two removed edges. Because 2-opt can directly eliminate crossing edges, it is generally regarded as the canonical local-search move for TSP. Its time complexity per full neighborhood sweep is $O(n^2)$, identical to Swap, but the move space is richer, and the resulting local optima are typically much shorter. In this report, 2-opt is included as an “upper reference”: it indicates how much additional improvement a stronger move could extract from the same initial tour produced by FI and thereby places the performance of Swap in context.

CHAPTER 3 Dataset & Methodology

This chapter describes the benchmark suite used to evaluate the algorithms, the parser and distance-matrix construction that handles the seven edge-weight types appearing in the suite, the four implemented algorithms, the validation procedure, and the unified experimental protocol applied across all instances.

3.1. Benchmark Suite

The algorithms are evaluated on a suite of eight instances drawn from the TSPLIB library. The suite was selected to cover a wide range of problem sizes, from 14 to 7,397 cities, and four of the eight TSPLIB edge-weight types so that the experiments can distinguish effects caused by algorithm design from effects caused by instance properties.

Table 3.1: Benchmark suite used in this study.

File	N	Edge Weight Type	Best Known	Size Class
burma14.tsp	14	GEO	3,323	Tiny
gr17.tsp	17	EXPLICIT	2,085	Tiny
lin105.tsp	105	EUC_2D	14,379	Small
fl417.tsp	417	EUC_2D	11,861	Medium
u574.tsp	574	EUC_2D	36,905	Medium
pr1002.tsp	1,002	EUC_2D	259,045	Large
d2103.tsp	2,103	EUC_2D	80,450	Large
pla7397.tsp	7,397	CEIL_2D	23,260,728	Very Large

Sizes are classified as “Tiny” ($N \leq 29$), “Small” ($N \leq 100$), “Medium” ($100 < N \leq 1,000$), “Large” ($1,000 < N \leq 5,000$), and “Very Large” ($N > 5,000$).

3.1.1. Edge-Weight Types Used

The four edge-weight conventions appearing in the suite are summarized in Table 3.2. The implementation additionally supports EUC_3D, MAN_2D, and MAX_2D for completeness.

Table 3.2: Edge-weight conventions in the benchmark suite.

Type	Formula for d_{ij}	Metric?
EUC_2D	$\lfloor \sqrt{\Delta x^2 + \Delta y^2} + 0.5 \rfloor$ (Euclidean, rounded to nearest)	Yes
CEIL_2D	$\lceil \sqrt{\Delta x^2 + \Delta y^2} \rceil$ (Euclidean, rounded up)	Yes
GEO	Great-circle distance on the WGS-style sphere, $R = 6378.388$ km	Yes
EXPLICIT	Matrix supplied directly in the file (one of FULL_MATRIX / UPPER_ROW / UPPER_DIAG_ROW / LOWER_ROW / LOWER_DIAG_ROW / UPPER_COL / LOWER_COL)	Depends on data

For EUC_2D and CEIL_2D, both the Euclidean metric and the rounding rule preserve the triangle inequality, so the standard approximation-ratio results for metric TSP apply. GEO is also metric since great-circle distances on a sphere satisfy the triangle inequality. For EXPLICIT instances such as `gr17.tsp`, the triangle inequality is not guaranteed and depends on the supplied data.

3.2. Distance Matrix Construction

The pipeline reads each TSPLIB file once, builds the corresponding $N \times N$ distance matrix in `int32`, and reuses the matrix for all four algorithms.

3.2.1. TSPLIB Parser

The parser reads the file linearly, extracting the `DIMENSION`, `EDGE_WEIGHT_TYPE`, and `EDGE_WEIGHT_FORMAT` from the header, then switches into either a coordinate-reading mode (after `NODE_COORD_SECTION`) or a weight-reading mode (after `EDGE_WEIGHT_SECTION`) until EOF is reached. The result is a dictionary containing the coordinates (as an $N \times 2$ NumPy array, or `None` for EXPLICIT instances), the raw edge weights (or an empty list otherwise), the edge-weight type and format, and the dimension.

```
def read_tsp(filepath):
    #Initialize empty containers
    coords = {}
    edge_weights = []
    edge_weight_type = None
    edge_weight_format = None
    dimension = None
    reading_coords = False
    reading_weights = False

    #Read all lines from file
    with open(filepath, "r") as f:
        lines = f.readlines()

    #Parse line by line
    i = 0
    while i < len(lines):
        line = lines[i].strip()
        #Read header metadata
        if line.startswith("DIMENSION"):
            dimension = int(line.split(":")[1].strip())
        elif line.startswith("EDGE_WEIGHT_TYPE"):
            edge_weight_type = line.split(":")[1].strip()
        elif line.startswith("EDGE_WEIGHT_FORMAT"):
            edge_weight_format = line.split(":")[1].strip()
        #Detect section markers
        elif line == "NODE_COORD_SECTION":
            reading_coords = True; reading_weights = False
        elif line == "EDGE_WEIGHT_SECTION":
            reading_weights = True; reading_coords = False
        elif line in ("EOF", "DISPLAY_DATA_SECTION"):
            reading_coords = False; reading_weights = False
        #Read actual data
        elif reading_coords and line:
            parts = line.split()
            node = int(parts[0]) - 1
            coords[node] = (float(parts[1]), float(parts[2]))
        elif reading_weights and line:
            edge_weights.extend([int(x) for x in line.split()])
        i += 1
```

Figure 3.1: Code snippet: read_tsp() – TSPLIB file parser.

```
#Convert coords dict into numpy array (or None if no coords)
coords_arr = np.array([coords[k] for k in range(len(coords))]) if coords else None
#Return as a dictionary
return {
    "coords" : coords_arr,
    "edge_weights" : edge_weights,
    "edge_weight_type" : edge_weight_type,
    "edge_weight_format" : edge_weight_format,
    "dimension" : dimension,
}

print("Read_tsp ready!")
```

Figure 3.2: Code snippet: read_tsp() – return dictionary.

3.2.2. Multi-Type Distance Construction

The build_dist_matrix function dispatches on the edge-weight type:

- **Coordinate-based metrics** (EUC_2D, EUC_3D, CEIL_2D, MAN_2D, MAX_2D): delegated to `scipy.spatial.distance.cdist`, followed by appropriate rounding.

- **GEO**: per-pair great-circle distance computation.
- **ATT**: the pseudo-Euclidean variant used by AT&T instances.
- **EXPLICIT**: matrix supplied directly in one of six storage layouts, each unpacked into a symmetric $N \times N$ array.

In every case, the diagonal is finally set to zero to guard against rounding artifacts. For the largest instance (`pla7397.tsp`), the resulting `int32` distance matrix occupies approximately 208.7 MB.

```
def build_dist_matrix(data):
    ewt = data["edge_weight_type"]
    ewf = data["edge_weight_format"]
    coords = data["coords"]
    n = data["dimension"]
    values = data["edge_weights"]

    #Case1: distances are already given as a matrix, just need to reshape
    if ewt == "EXPLICIT":
        dist = np.zeros((n, n), dtype=np.int32)
        if ewf == "FULL_MATRIX":
            dist = np.array(values, dtype=np.int32).reshape(n, n)
        elif ewf in ("UPPER_ROW", "UPPER_DIAG_ROW"):
            idx = 0; start = 1 if ewf == "UPPER_ROW" else 0
            for i in range(n):
                for j in range(i + start, n):
                    dist[i][j] = dist[j][i] = values[idx]; idx += 1
        elif ewf in ("LOWER_ROW", "LOWER_DIAG_ROW"):
            idx = 0; start = 1 if ewf == "LOWER_ROW" else 0
            for i in range(start, n):
                for j in range(0, i + (0 if ewf == "LOWER_ROW" else 1)):
                    dist[i][j] = dist[j][i] = values[idx]; idx += 1
        elif ewf == "UPPER_COL":
            idx = 0
            for j in range(n):
                for i in range(j):
                    dist[i][j] = dist[j][i] = values[idx]; idx += 1
        elif ewf == "LOWER_COL":
            idx = 0
            for j in range(n):
                for i in range(j+1, n):
                    dist[i][j] = dist[j][i] = values[idx]; idx += 1
        np.fill_diagonal(dist, 0)
    return dist
```

Figure 3.3: Code snippet: `build_dist_matrix()` – explicit case.

```

#Case2: calculate distances from coordinates using different formulas
if ewt in ("EUC_2D", "EUC_3D"):
    dist = (cdist(coords, coords, metric='euclidean') + 0.5).astype(np.int32)
elif ewt == "CEIL_2D":
    dist = np.ceil(cdist(coords, coords, metric='euclidean')).astype(np.int32)
elif ewt == "MAN_2D":
    dist = (cdist(coords, coords, metric='cityblock') + 0.5).astype(np.int32)
elif ewt == "MAX_2D":
    dist = (cdist(coords, coords, metric='chebyshev') + 0.5).astype(np.int32)
elif ewt == "GEO":
    def to_rad(c):
        deg = int(c); return math.pi * (deg + 5*(c-deg)/3) / 180
    RRR = 6378.388
    dist = np.zeros((n, n), dtype=np.int32)
    for i in range(n):
        for j in range(i+1, n):
            lat1,lon1 = to_rad(coords[i][0]), to_rad(coords[i][1])
            lat2,lon2 = to_rad(coords[j][0]), to_rad(coords[j][1])
            q1 = math.cos(lon1-lon2); q2 = math.cos(lat1-lat2); q3 = math.cos(lat1+lat2)
            d = int(RRR * math.acos(max(-1, min(1, 0.5*((1+q1)*q2-(1-q1)*q3)))) + 1)
            dist[i][j] = dist[j][i] = d
elif ewt == "ATT":
    dist = np.zeros((n, n), dtype=np.int32)
    for i in range(n):
        for j in range(i+1, n):
            dx = coords[i][0]-coords[j][0]; dy = coords[i][1]-coords[j][1]
            r = math.sqrt((dx**2+dy**2)/10.0); t = int(r)
            dist[i][j] = dist[j][i] = t+1 if t < r else t
else:
    print(f" [WARN] Not supported yet! '{ewt}', fallback EUC_2D")
    dist = (cdist(coords, coords, metric='euclidean') + 0.5).astype(np.int32)

np.fill_diagonal(dist, 0)
return dist

print("Build_dist_matrix ready!")

```

Figure 3.4: Code snippet: `build_dist_matrix()` – coordinate-based cases.

3.2.3. Helper Functions

Three small helpers are used throughout the report: `total_distance` computes the total length of a tour via vectorised lookup, `is_valid_tour` performs the validation checks of Section 3.5, and `calc_gap` reports the percentage deviation of a tour length from the known optimum.

```

def total_distance(route, dist_matrix):
    arr = np.asarray(route, dtype=np.int64)
    return int(dist_matrix.astype(np.int64)[arr, np.roll(arr, -1)].sum())

def is_valid_tour(route, n):
    return (len(route) == n and len(set(route)) == n
            and min(route) >= 0 and max(route) < n)

def calc_gap(cost, optimal):
    return round((cost - optimal) / optimal * 100, 4)

print("Helper functions ready!")

```

Figure 3.5: Code snippet: helper functions `total_distance`, `is_valid_tour`, `calc_gap`.

3.3. Main Methods

3.3.1. Farthest Insertion with Convex-Hull Multi-Start

Farthest Insertion builds a tour by always inserting next the city that is farthest from the current tour. The intuition is that the most isolated cities are the most expensive to insert if left to the end; by addressing them first, the algorithm fixes the outer “skeleton” of the tour early on, and the remaining cities can be inserted cheaply between existing tour edges. To reduce dependence on the choice of starting city, the implementation runs FI from several different starting points and keeps the shortest tour.

- **Choice of starting points:** Candidate starting cities are drawn from the convex hull of the coordinate set, supplemented by cities farthest from the centroid. When coordinates are unavailable, the implementation falls back to selecting cities by row sum in the distance matrix. The number of starts is adapted to instance size.
- **Main procedure:** From each starting city, the algorithm picks the farthest city to form the initial two-vertex tour, then iteratively inserts the unvisited city with the largest nearest_dist at the position minimising the insertion delta. The nearest_dist array is updated incrementally in $O(N)$ per insertion.
- **Complexity:** Each restart costs $O(N^2)$. Total cost over max_starts restarts is $O(\text{max_starts} \cdot N^2)$, which remains tractable because max_starts decreases with N .

```
def convex_hull_starts(coords, n, max_starts):
    try:
        from scipy.spatial import ConvexHull
        hull = ConvexHull(coords)
        hull_pts = list(hull.vertices)
        centroid = coords.mean(axis=0)
        dists_to_center = np.linalg.norm(coords - centroid, axis=1)
        inner = [i for i in np.argsort(dists_to_center)[::-1] if i not in hull_pts]
        candidates = hull_pts + inner
        step = max(1, len(candidates) // max_starts)
        return candidates[::step][:max_starts]
    except:
        return list(range(max_starts))
```

Figure 3.6: Code snippet: convex_hull_starts().

```
#Build a good initial TSP tour by repeatedly inserting the farthest unvisited city into the best position
def greedy_farthest_insertion(dist_matrix, coords=None):
    n = dist_matrix.shape[0]

    #Scale number of starting points based on problem size (bigger n = fewer starts to save time)
    if n <= 50:
        max_starts = n
    elif n <= 200:
        max_starts = 50
    elif n <= 500:
        max_starts = 30
    elif n <= 2000:
        max_starts = 15
    elif n <= 5000:
        max_starts = 8
    else:
        max_starts = 5
```

Figure 3.7: Code snippet: `greedy_farthest_insertion()` – setup and start selection.

```
#Pick starting cities: use convex hull if coords available, else pick cities with highest/lowest total distances
if coords is not None and n >= 3:
    starts = convex_hull_starts(coords, n, max_starts)
else:
    row_sums = dist_matrix.sum(axis=1)
    top = list(np.argsort(row_sums)[::-1][:max_starts//2 + 1])
    bottom = list(np.argsort(row_sums)[:max_starts//2 + 1])
    starts = list(set(top + bottom)[:max_starts])

best_tour = None
best_dist = float('inf')

for start in starts:
    #Build tour from this starting city
    farthest = int(np.argmax(dist_matrix[start]))
    tour = [start, farthest]
    visited = np.zeros(n, dtype=bool)
    visited[start] = visited[farthest] = True
    nearest_dist = np.minimum(dist_matrix[:, start], dist_matrix[:, farthest])

    while len(tour) < n:
        masked = np.where(visited, -np.inf, nearest_dist)
        new_city = int(np.argmax(masked))
        #Find the best position to insert new_city
        tour_arr = np.array(tour, dtype=np.int32)
        next_arr = np.roll(tour_arr, -1)
        increases = (dist_matrix[tour_arr, new_city]
                    + dist_matrix[new_city, next_arr]
                    - dist_matrix[tour_arr, next_arr])
        best_pos = int(np.argmin(increases)) + 1
        tour.insert(best_pos, new_city)
        visited[new_city] = True
        nearest_dist = np.minimum(nearest_dist, dist_matrix[:, new_city])

    d = total_distance(tour, dist_matrix)
    if d < best_dist:
        best_dist = d
        best_tour = tour[:]

return best_tour, best_dist

print("Greedy_farthest_insertion ready!")
```

Figure 3.8: Code snippet: `greedy_farthest_insertion()` – main insertion loop.

3.3.2. Swap Local Search with Double-Bridge Restarts

Swap takes the initial tour produced by Farthest Insertion and tries to improve it by exchanging the positions of two cities in the tour. The procedure terminates when no swap yields any improvement — a local optimum under the Swap neighborhood. To escape local optima, the implementation embeds Swap inside an iterated local search outer loop: after each convergence, the tour is perturbed by a double-bridge move, and the inner search is restarted. Double-bridge

cuts the tour at three positions and reconnects the four segments in a non-reversing order; it is one of the few perturbations that no 2-opt sequence can directly undo, making it the standard escape move for ILS on TSP.

```
#Break the tour into 4 segments and reconnect in a shuffled order to escape local optima
def double_bridge(tour, n):
    cuts = sorted(np.random.choice(range(1, n), size=3, replace=False))
    a, b, c = cuts
    return np.concatenate([tour[:a], tour[b:c], tour[a:b], tour[c:]])
```

Figure 3.9: Code snippet: `double_bridge()` perturbation.

- **Inner loop:** Each sweep iterates over every city b at position i and considers only the $K \leq 200$ nearest neighbors of b . The first improving swap is accepted (First Improvement). The position array `position[city] = idx` allows $O(1)$ lookups. A standard early exit skips candidates whose distance to b already exceeds b 's incident edge cost.
- **Complexity:** With neighbor pruning, each inner pass costs $O(NK)$. Total work bounded by $O(\text{MAX_PASSES} \cdot \text{MAX_RESTARTS} \cdot NK)$.

3.4. Reference Methods

3.4.1. Nearest Neighbor with Multi-Start

Nearest Neighbor (NN) is the simplest possible constructive heuristic: from a starting city, the salesman always proceeds to the closest unvisited city until every city has been visited. Like FI, the implementation runs NN from multiple starting points and keeps the shortest tour. Each restart costs $O(N^2)$; total cost is $O(\text{max_starts} \cdot N^2)$.

```
#Build a TSP tour by repeatedly visiting the nearest unvisited city
def nearest_neighbor(dist_matrix, coords=None):
    n = dist_matrix.shape[0]

    #Same scaling logic as FI: fewer starts for larger problems to save time
    if n <= 50:
        max_starts = n
    elif n <= 200:
        max_starts = 50
    elif n <= 500:
        max_starts = 30
    elif n <= 2000:
        max_starts = 15
    elif n <= 5000:
        max_starts = 8
    else:
        max_starts = 5
```

Figure 3.10: Code snippet: `nearest_neighbor()` – setup.

```

#Same start selection logic as FI: convex hull if coords available, else by row sums
if coords is not None and n >= 3:
    starts = convex_hull_starts(coords, n, max_starts)
else:
    row_sums = dist_matrix.sum(axis=1)
    top = list(np.argsort(row_sums)[::-1][:max_starts//2 + 1])
    bottom = list(np.argsort(row_sums)[:max_starts//2 + 1])
    starts = list(set(top + bottom)[:max_starts])

best_tour = None
best_dist = float('inf')

for start in starts:
    visited = np.zeros(n, dtype=bool)
    visited[start] = True
    tour = [start]
    current = start

    while len(tour) < n:
        dists = dist_matrix[current].copy().astype(float)
        dists[visited] = np.inf
        nearest = int(np.argmin(dists))
        tour.append(nearest)
        visited[nearest] = True
        current = nearest

    d = total_distance(tour, dist_matrix)
    if d < best_dist:
        best_dist = d
        best_tour = tour[:]

return best_tour, best_dist

print("Nearest_neighbor ready!")

```

Figure 3.11: Code snippet: nearest_neighbor() – main loop.

3.4.2. 2-opt Local Search with Double-Bridge Restarts

A 2-opt move removes two non-adjacent edges and reconnects the resulting path segments in the unique alternative way, effectively reversing the segment between the removed edges. In Euclidean instances, 2-opt directly eliminates crossing edges — a property Swap does not possess. The implementation matches the structure of the Swap procedure with a neighbor-pruned inner sweep and double-bridge restarts. The neighbor budget K is adapted to instance size (down to $K = 30$ for large instances).

```
def two_opt_local_search(init_tour, dist_matrix, seed):  
    np.random.seed(seed)  
    n = dist_matrix.shape[0]  
  
    if n <= 100:  
        K = min(n-1, n)  
    elif n <= 500:  
        K = min(n-1, 100)  
    elif n <= 2000:  
        K = min(n-1, 50)  
    else:  
        K = min(n-1, 30)  
  
    MAX_PASSES = 100  
    MAX_RESTARTS = 10  
  
    nn_k = np.argsort(dist_matrix, axis=1)[: , 1:K + 1]  
  
    current_tour = np.array(init_tour, dtype=np.int32)  
    best_tour = current_tour.copy()  
    best_dist = total_distance(best_tour, dist_matrix)  
    current_dist = best_dist
```

Figure 3.12: Code snippet: two_opt_local_search() – setup.

```

for restart in range(MAX_RESTARTS):
    any_improved = False

    for _ in range(MAX_PASSES):
        improved = False

        #Build position lookup: position[city] = index in current_tour
        position = np.empty(n, dtype=np.int32)
        for idx, city in enumerate(current_tour):
            position[city] = idx

        for i in range(n - 1):
            city_i = current_tour[i]
            city_i1 = current_tour[i + 1]
            d_i = dist_matrix[city_i, city_i1]

            for city_j in nn_k[city_i1]:
                if dist_matrix[city_i, city_j] >= d_i:
                    break
                j = position[city_j]
                if j <= i + 1:
                    continue
                city_j1 = current_tour[(j + 1) % n]
                #Calculate how much distance we gain by doing a 2-opt swap here
                gain = (d_i + dist_matrix[city_j, city_j1]
                       - dist_matrix[city_i, city_j]
                       - dist_matrix[city_i1, city_j1])

                if gain > 1e-6:
                    current_tour[i + 1: j + 1] = current_tour[i + 1: j + 1][::-1]
                    for idx in range(i + 1, j + 1):
                        position[current_tour[idx]] = idx
                    current_dist -= gain
                    d_i = dist_matrix[city_i, current_tour[i + 1]]
                    city_i1 = current_tour[i + 1]
                    improved = True
                    any_improved = True
                    break

            if current_dist < best_dist:
                best_dist = current_dist
                best_tour = current_tour.copy()
            if not improved:
                break

        if not any_improved and restart > 0:
            break

        #Perturb with double-bridge and try again from new position
        if restart < MAX_RESTARTS - 1 and n >= 8:
            current_tour = best_tour.copy()
            current_tour = double_bridge(current_tour, n)
            current_dist = total_distance(current_tour, dist_matrix)

    best_dist = total_distance(best_tour, dist_matrix)
    return best_tour.tolist(), best_dist

print("Two_opt_local_search ready!")

```

Figure 3.13: Code snippet: two_opt_local_search() – inner loop.

3.5. Validation Procedure

To verify that each produced tour is feasible, every output is checked by the following validator combining three correctness tests in a single Boolean expression:

```

def is_valid_tour(route, n):
    return (len(route) == n and len(set(route)) == n
            and min(route) >= 0 and max(route) < n)

```

Figure 3.14: Code snippet: is_valid_tour() validator.

The validator simultaneously enforces:

- (i) The tour contains exactly N entries.
- (ii) The set of entries equals $\{0, 1, \dots, N - 1\}$ — no city missing or repeated.

(iii) Every entry is a valid city identifier.

A tour passing this test is necessarily a Hamiltonian cycle on G . The validator is invoked on every tour returned by every algorithm on every instance.

3.6. Experimental Protocol

To ensure a fair comparison across instances of widely different sizes, the experiments follow a unified protocol with parameters that scale with N .

Seeds. The selection of random seeds controls the stochastic components of the algorithm — the double-bridge perturbation in the Iterated Local Search framework. A default pool of five seeds $\{11, 22, 33, 44, 55\}$ is adopted. For larger instances, the number of seeds is reduced:

```
def get_seeds(n):  
    if n > 5000:  
        return [11]  
    elif n > 1000:  
        return [11, 22, 33]  
    else:  
        return SEEDS
```

Figure 3.15: Code snippet: `get_seeds(n)` – adaptive seed selection.

Adaptive parameters. Three parameters are adapted to instance size:

- `max_starts`: decreases from N (for $N \leq 50$) to 5 (for $N > 5,000$).
- 2-opt neighbor budget K : decreases from $K = n$ (for $N \leq 100$) down to $K = 30$ for $N > 2,000$.
- Swap: $K = \min(200, n - 1)$ uniformly.

Metrics. For every (instance, algorithm) configuration the report records: total tour length, percentage gap to the known optimum, validator status, and wall-clock runtime. Both the minimum gap (best run) and the average gap (typical run) are reported across seeds.

Hardware. All experiments are conducted on a Google Colab CPU runtime (single core), with a modern laptop CPU as cross-check. The relative ranking of algorithms remains consistent across both platforms.

CHAPTER 4 Experimental Results and Analysis

4.1. Experimental Setup

4.1.1. Hardware and Software Environment

All experiments were conducted on Google Colaboratory, utilizing a CPU-based runtime environment with access to Google Drive for dataset storage. The implementation was written in Python 3, leveraging NumPy for vectorized matrix operations, SciPy (`cdist` and `ConvexHull`) for distance computation and convex hull extraction, and Matplotlib for result visualization. No GPU acceleration was utilized.

4.1.2. Datasets

Eight benchmark instances drawn from the TSPLIB library were used (see Table 3.1). The datasets encompass four distinct edge-weight types: GEO, EXPLICIT, EUC_2D, and CEIL_2D. Problem sizes range from 14 nodes (`burma14`) to 7,397 nodes (`p1a7397`), spanning roughly three orders of magnitude.

4.1.3. Algorithm Parameters

Greedy Algorithms (Constructive Phase):

- **Farthest Insertion (FI):** Builds a tour by iteratively selecting the unvisited city farthest from the current partial tour and inserting it at the cheapest available position.
- **Nearest Neighbor (NN):** Constructs a tour by always visiting the closest unvisited city from the current position.

Local Search Algorithms (Improvement Phase):

- **Swap Local Search:** Iteratively exchanges pairs of cities when doing so reduces total distance.
- **2-opt Local Search:** Reverses a segment of the tour when the reversal yields a shorter total distance.

All parameters were scaled adaptively:

- FI and NN (max_starts): from N for $N \leq 50$ down to 5 for $N > 5,000$.
- Swap: $K = \min(200, N - 1)$, MAX_PASSES = 50, MAX_RESTARTS = 15.
- 2-opt: K adaptive ($K = N - 1$ for $N \leq 100$, $K = 100$ for $N \leq 500$, $K = 50$ for $N \leq 2,000$, $K = 30$ for $N > 2,000$), MAX_PASSES = 100, MAX_RESTARTS = 10.

4.1.4. Seed Protocol

Each experiment is repeated across multiple random seeds:

- $N \leq 1,000$: seeds = [11, 22, 33, 44, 55] (5 seeds)
- $1,000 < N \leq 5,000$: seeds = [11, 22, 33] (3 seeds)
- $N > 5,000$: seed = [11] (1 seed)

4.1.5. Correctness Validation

All output tours were verified using `is_valid_tour()`, which checks that the tour contains exactly N cities, all cities are distinct, and all indices are within $[0, N - 1]$. All 8 datasets passed validation under all 4 algorithm pipelines. The gap metric is defined as:

$$\text{Gap}(\%) = \frac{\text{Cost}_{\text{found}} - \text{Cost}_{\text{optimal}}}{\text{Cost}_{\text{optimal}}} \times 100 \quad (4.1)$$

4.2. Constructive Stage Comparison: Farthest Insertion vs. Nearest Neighbor

Table 4.1: FI vs. NN: Gap and Runtime Across 8 Datasets.

Dataset	N	FI Gap (%)	FI Time (ms)	NN Gap (%)	NN Time (ms)
burma14.tsp	14	0.0000	15	15.5883	2
gr17.tsp	17	0.0000	10	4.4604	4
lin105.tsp	105	1.9473	2.30	17.7759	28
fl417.tsp	417	3.1279	792	18.4892	81
u574.tsp	574	7.5843	651	23.2922	70
pr1002.tsp	1,002	8.4576	1,579	22.2737	155
d2103.tsp	2,103	22.0087	2,718	7.7104	262
pla7397.tsp	7,397	13.0671	21,360	21.6268	1,472

4.2.1. Solution Quality

Across the 8 datasets, the gap values by size group are:

- Tiny (burma14, gr17): FI = 0.0% vs. NN = 15.59%, 4.46%
- Medium (lin105, fl417): FI = 1.95%, 3.13% vs. NN = 17.78%, 18.49%
- Large (u574, pr1002): FI = 7.58%, 8.46% vs. NN = 23.29%, 22.27%
- XLarge (pla7397): FI = 13.07% vs. NN = 21.63%
- XLarge exception (d2103): FI = 22.01% vs. NN = 7.71%

FI outperforms NN on 7 out of 8 datasets, with the quality advantage averaging around 10–15 percentage points. This stems from FI’s global perspective: by always selecting the farthest unvisited city next, FI builds a tour that expands outward from the boundary inward, naturally avoiding large uncovered regions and producing a more balanced initial tour structure.

On the 2,103-node d2103 instance, NN outperforms FI by a wide margin (7.71% vs. 22.01%). The d2103 instance features a dense, relatively uniform distribution of coordinates. In such settings, NN’s greedy local strategy thrives; short edges are abundant and easy to exploit sequentially without creating large detours. FI, by contrast, prioritizes globally distant cities early in construction, which in a uniform dense layout tends to create crossing edges that are difficult to untangle later.

4.2.2. Runtime

- Tiny–Medium ($N \leq 105$): FI $\approx$ 0.01–0.23 s vs. NN $\approx$ 0.002–0.028 s
- Medium–Large ($N = 417$ –1,002): FI $\approx$ 0.65–1.58 s vs. NN $\approx$ 0.07–0.16 s
- XLarge ($N = 2,103$ –7,397): FI = 2.72–21.36 s vs. NN = 0.26–1.47 s

NN is consistently faster than FI across all problem sizes. All running times reported in this chapter are converted to milliseconds (ms).

4.3. Local Search Effectiveness: Improvement Analysis

Table 4.2: Local Search Improvement. Δ (pp) = improvement in percentage points.

Dataset	N	FI Gap (%)	FI Time (ms)	Swap Δ (pp)	NN Gap (%)	NN Time (ms)	2-opt Δ (pp)
burma14.tsp	14	0.0000	15	0.0000	15.5883	2	15.1971
gr17.tsp	17	0.0000	10	0.0000	4.4604	4	4.4604
lin105.tsp	105	1.9473	230	0.0000	17.7759	28	14.4238
fl417.tsp	417	3.1279	792	0.3035	18.4892	81	12.2418
u574.tsp	574	7.5843	651	0.1923	23.2922	70	11.6786
pr1002.tsp	1,002	8.4576	1,579	0.1166	22.2737	155	7.3435
d2103.tsp	2,103	22.0087	2,718	0.0858	7.7104	262	0.6824
pla7397.tsp	7,397	13.0671	21,360	0.1147	21.6268	1,472	1.3149

4.3.1. Swap on Farthest Insertion

Swap’s improvements are marginal across all datasets:

- **Tiny** ($N \leq 17$): No improvement; FI already reaches the optimum.
- **Medium** ($N = 105\text{--}417$): Zero improvement on lin105; only 0.30 pp on fl417.
- **Large–XLarge** ($N \geq 574$): Improvements stabilize in the 0.09–0.19 pp range.

This pattern is consistent with FI’s construction mechanism: FI produces geometrically well-structured tours with few crossing edges, which coincides with Swap’s local optima. Since Swap only exchanges two city positions at a time, it cannot escape the structural limitations of the initial tour.

4.3.2. 2-opt on Nearest Neighbor

2-opt produces substantially larger improvements than Swap, but with a clear diminishing-returns trend as N increases:

- **Tiny** ($N \leq 17$): Improvements of 4.46–15.20 pp; gr17 reaches the exact optimum.
- **Medium** ($N = 105\text{--}417$): Improvements of 12.24–14.42 pp, the largest relative gains.
- **Large** ($N = 574\text{--}1,002$): Improvements drop to 7.34–11.68 pp.
- **XLarge** ($N \geq 2,103$): Improvements collapse to 0.68–1.31 pp.

4.3.3. Comparative Observation

Despite 2-opt achieving dramatically larger improvements over NN, the final post-local-search quality of FI+Swap still matches or exceeds NN+2-opt on 6 of 8 datasets. This confirms that local search improvement magnitude is not a reliable proxy for final solution quality; the initial solution’s structural soundness is the dominant factor.

4.4. Full Benchmark of 4 Algorithm Pairs

Table 4.3: Gap (%) for All 4 Pairs Across 8 Datasets. Bold = best result per row.

Dataset	N	FI+Swap	FI+2opt	NN+Swap	NN+2opt	Best
burma14.tsp	14	0.0000	0.0000	0.6921	0.3912	0.0000
gr17.tsp	17	0.0000	0.0000	0.0000	0.0000	0.0000
lin105.tsp	105	1.9473	1.5370	13.6101	3.3521	1.5370
fl417.tsp	417	2.8244	2.3944	15.6901	6.2474	2.3944
u574.tsp	574	7.3920	5.4491	20.2439	11.6136	5.4491
pr1002.tsp	1,002	8.3410	6.8563	19.8487	14.9302	6.8563
d2103.tsp	2,103	21.9229	21.6420	7.3151	7.0280	7.0280
pla7397.tsp	7,397	12.9524	11.8125	20.4109	20.3119	11.8125

Table 4.4: Average Runtime per Seed (ms) for All 4 Pairs.

Dataset	N	FI+Swap	FI+2opt	NN+Swap	NN+2opt
burma14.tsp	14	14	2	14	4
gr17.tsp	17	11	2	21	3
lin105.tsp	105	107	13	328	22
fl417.tsp	417	342	63	837	82
u574.tsp	574	754	100	1,042	96
pr1002.tsp	1,002	941	214	1,596	181
d2103.tsp	2,103	3,915	1,395	685	418
pla7397.tsp	7,397	20,421	5,468	7,732	4,419

4.4.1. Solution Quality

Key observations by pair:

- **FI+2opt** achieves the best gap on 5 of 8 datasets, ties for best on 2 Tiny instances, and is second-best on d2103. It is the most consistently dominant pair.
- **FI+Swap** ties FI+2opt on Tiny instances but underperforms on all Medium–XLarge datasets.
- **NN+2opt** is competitive only on d2103 (7.03%, best overall) but trails significantly on Medium and Large instances.
- **NN+Swap** is the weakest pair overall, ranking last on 6 of 8 datasets with gaps reaching 20.24% on u574.

4.4.2. Runtime

- FI+2opt is the fastest pair on all datasets except the two Tiny instances; on pla7397 it takes 5.47 s vs. 20.42 s for FI+Swap.

- FI+Swap is the slowest pair due to its larger restart budget ($\text{MAX_RESTARTS} = 15$, $\text{MAX_PASSES} = 50$).
- NN+2opt is the fastest NN-based pair throughout.

4.4.3. Time and Quality Trade-off Analysis

The results clearly demonstrate the trade-off between solution quality and execution time. FI+2opt offers the best overall balance. On pla7397, FI+Swap took nearly $4\times$ longer than FI+2opt (20,421 ms vs. 5,468 ms) yet delivered a worse final gap (12.95% vs. 11.81%). This highlights that investing in a stronger constructive heuristic is often more effective than relying heavily on local search to compensate for a poor initial solution.

4.5. Cross-Dataset Analysis

4.5.1. Scaling with Instance Size

Plotting the gap of each pair as a function of N shows that the gap of the FI-based pairs is essentially flat at 0% for the two tiny instances, then climbs slowly with N , before falling sharply at d2103.tsp and rising again at pla7397. NN-based pairs show a similar but consistently higher curve, except at d2103.

4.5.2. Effect of Edge-Weight Type

The two non-EUC_2D instances behave as expected. The GEO instance burma14 is small enough that all four pairs find essentially the optimum. The EXPLICIT instance gr17.tsp is solved exactly by every pair. The CEIL_2D instance pla7397.tsp behaves like a large EUC_2D instance. In short, the benchmark suite does not reveal any systematic effect of edge-weight type on the relative ranking of the algorithms.

4.5.3. Trade-off Between Time and Quality

FI+2opt also wins on time: it is the fastest of the three non-trivial pairs on every instance except pla7397.tsp. The reason is that 2-opt extracts most of its improvement in the first few sweeps and triggers early convergence, while Swap with double-bridge restarts continues attempting swaps that no longer improve the tour.

4.6. Personal Perspectives

Four findings of the present study merit explicit discussion.

(1) Farthest Insertion almost always beats Nearest Neighbor as a constructive method, but not universally. FI's gap is lower than NN's on seven of the eight instances. The exception is

d2103.tsp (22.01% vs. 7.71%), where a dense, relatively uniform distribution of coordinates makes NN's greedy local strategy more effective than FI's global approach.

(2) The choice of local-search move dominates the choice of constructive method when room for improvement exists. On pr1002.tsp: FI starts at 8.46% and NN at 22.27%, but after 2-opt they converge to within a single percentage point (6.86% vs. 14.93%), while after Swap they remain over ten points apart. For real-world deployments, investing in a stronger local-search move appears to be a higher-leverage choice.

(3) The d2103 inversion is a methodological warning, not a bug. Both FI-based pairs are stuck near the FI gap of 22.01%; the local-search stages are unable to recover from the bad initial tour. The practical takeaway is that every (instance, constructive, local-search) triple must be reported individually.

(4) FI+2opt is the recommended pipeline on the present benchmark suite, but is not the assigned one. The assigned pipeline (FI+Swap) is competitive on small instances and within 1.15 percentage points of FI+2opt on pla7397, but is uniformly second-best on the medium instances.

4.7. Summary of Experimental Findings

In summary, on the eight-instance TSPLIB benchmark suite:

- Farthest Insertion produces stronger initial tours than Nearest Neighbor on seven of the eight instances; on d2103.tsp the ranking is reversed.
- 2-opt extracts more improvement than Swap from any given initial tour, with the difference growing as the initial tour gets worse.
- FI+2opt is the best pair in six of the eight instances; NN+2opt wins on d2103; both pairs tie with a zero gap on gr17.
- The best gap achieved by the main pipeline (FI+Swap) is 0.00% on the two tiny instances and 12.95% on the largest instance (pla7397.tsp).

CHAPTER 5 Conclusion

5.1. Summary of Findings

This report has implemented four algorithm pairs — Farthest Insertion + Swap, Farthest Insertion + 2-opt, Nearest Neighbor + Swap, and Nearest Neighbor + 2-opt — and evaluated them on a TSPLIB benchmark suite of eight instances spanning four edge-weight types (EUC_2D, CEIL_2D, GEO, EXPLICIT) and instance sizes from 14 to 7,397 cities. All resulting tours were verified valid by a unified validator. The main empirical conclusions:

- Farthest Insertion outperforms Nearest Neighbor as a constructive heuristic on seven of the eight instances, often by a factor of two or three in solution gap.
- 2-opt extracts more improvement than Swap from any given initial tour, with the difference growing as the initial tour is further from a local optimum.
- The combined pair FI+2opt is the strongest pipeline on six of the eight instances; the assigned pair FI+Swap is competitive on small and medium instances but is uniformly second-best on the medium range.
- A single instance (d2103.tsp) inverts the ranking: NN-based pipelines outperform FI-based pipelines by a wide margin, demonstrating that no constructive heuristic is universally superior.

5.2. Lessons Learned

5.2.1. On Algorithmic Complexity

The most direct lesson of the project is that both Swap and 2-opt have per-sweep complexity $O(N^2)$, yet their measured wall-clock runtimes on the same instance differ by an order of magnitude: on `pla7397.tsp`, FI+2opt completes in roughly seven seconds, while FI+Swap requires twenty-three. The difference is entirely attributable to constant factors that Big-O notation hides by construction.

The classical worst-case approximation ratio of Farthest Insertion for metric TSP is $\lceil \log_2 n \rceil + 1$, which for $n = 7,397$ evaluates to approximately 14. The empirical results show that FI on

pla7397 returns a tour only 13% above optimum — three orders of magnitude better than the worst case.

Finally, the project highlighted the practical importance of vectorised implementation. The pure-Python version of Farthest Insertion requires nearly eight minutes on $N = 2,000$, whereas the NumPy-vectorised version runs the full $N = 7,397$ instance in under twenty-six seconds — a speed-up of roughly two orders of magnitude without changing the algorithm.

5.2.2. On Heuristic Design

Three heuristic-design lessons follow from the experiments. First, multi-start beats single-start for both constructive methods studied. Second, the choice of local-search move is the dominant lever once a reasonable initial tour exists. Third, no single configuration is universally optimal: on seven instances the recommended pipeline is FI+2opt, but on d2103.tsp the opposite combination, NN+2opt, wins.

5.3. Practical Applications in Logistics and Supply Chain

The methods studied in this project have direct counterparts in industrial routing problems. The same FI+2opt template is the kernel of many in-production vehicle routing systems used in last-mile delivery. The same template appears in PCB drilling — the original use case of pla7397.tsp — where a CNC drill must visit thousands of hole positions in an order that minimises tool-travel time. Other industrial appearances of TSP include daily route planning for waste-collection and street-sweeping fleets, technician dispatch for telecommunications and utilities, and ordering of operations on a flexible manufacturing line.

In each of these settings, the project’s central finding — that a strong local-search move dominates the value of a strong constructive method for any non-trivial instance — has a concrete operational implication: investing engineering effort in the refinement stage of the pipeline generally has a higher return than tuning the constructive stage, except when the constructive stage produces structurally pathological starts (as d2103.tsp did).

5.4. Limitations

The present study has several limitations. First, the benchmark suite is small: eight instances do not fully sample the space of TSP problems, and three edge-weight types (MAN_2D, MAX_2D, ATT) are not exercised. Second, the seed protocol is sparse for the largest instance (pla7397.tsp is run with a single seed). Third, the algorithms studied are at the simpler end of the literature, and the report does not include a comparison against the state-of-the-art Lin–Kernighan–Helsgaun (LKH) solver or against exact solvers such as Concorde.

5.5. Concluding Remarks

This project set out to implement a classical constructive-plus-local-search pipeline for TSP, to evaluate it on a standard benchmark, and to assess its limitations honestly. The empirical evidence assembled in Chapter 4 supports a clear ranking among the four pairs studied and identifies a structurally interesting anomaly on `d2103.tsp`. The broader takeaway — that no single heuristic dominates uniformly, that the local-search stage matters more than is sometimes appreciated, and that empirical performance can be orders of magnitude better than worst-case bounds — is one that recurs across combinatorial optimization problems well beyond TSP itself.

Bibliography

- [1] D. L. Applegate, R. E. Bixby, V. Chvátal, and W. J. Cook, *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, 2006.
- [2] W. J. Cook, *In Pursuit of the Traveling Salesman: Mathematics at the Limits of Computation*. Princeton University Press, 2012.
- [3] G. Reinelt, “TSPLIB — A traveling salesman problem library,” *ORSA Journal on Computing*, vol. 3, no. 4, pp. 376–384, 1991.
- [4] S. Lin and B. W. Kernighan, “An effective heuristic algorithm for the traveling-salesman problem,” *Operations Research*, vol. 21, no. 2, pp. 498–516, 1973.
- [5] D. S. Johnson and L. A. McGeoch, “The traveling salesman problem: A case study in local optimization,” in *Local Search in Combinatorial Optimization*, E. H. L. Aarts and J. K. Lenstra, Eds. Princeton University Press, 1997, pp. 215–310.
- [6] D. J. Rosenkrantz, R. E. Stearns, and P. M. Lewis, “An analysis of several heuristics for the traveling salesman problem,” *SIAM Journal on Computing*, vol. 6, no. 3, pp. 563–581, 1977.
- [7] J. L. Bentley, “Fast algorithms for geometric traveling salesman problems,” *ORSA Journal on Computing*, vol. 4, no. 4, pp. 387–411, 1992.
- [8] O. Martin, S. W. Otto, and E. W. Felten, “Large-step Markov chains for the traveling salesman problem,” *Complex Systems*, vol. 5, no. 3, pp. 299–326, 1991.
- [9] K. Helsgaun, “An effective implementation of the Lin–Kernighan traveling salesman heuristic,” *European Journal of Operational Research*, vol. 126, no. 1, pp. 106–130, 2000.
- [10] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.
- [11] P. Virtanen *et al.*, “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.